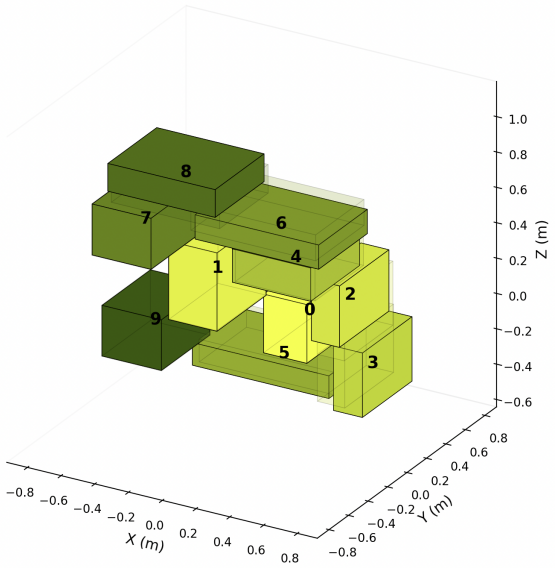
Master Semester Project

ETH Zürich, D-INFK

Generative Component Placement:

Architectures and Constraints for Differentiable Engineering Design



Phuc Vu

phucvu@ethz.ch

Supervisor:	Prof. Stelian Coros
Co-Supervisors:	Prof. Mark Fuge, Tim Aebersold
Laboratory:	Intelligence for Design, Engineering And Learning (IDEAL)
Period:	February – May 2026

June 5, 2026

Contents

1 Introduction **2**
1.1 Contributions . . . 2

2 Related Work **3**
2.1 Layout generation . . . 3
2.2 Differentiable design pipelines . . . 3

3 Problem Formulation **4**

4 Architectures **5**
4.1 MLP baseline . . . 5
4.2 Auto-regressive transformer . . . 6
4.3 Parallel decoder transformer . . . 7
4.4 Conditional variants . . . 8

5 Training Objectives **9**
5.1 Geometric losses . . . 9
5.2 Diversity: Aspect Ratios DPP and Layout Diversity . . . 10
5.3 InfoNCE . . . 10
5.4 Weighted-sum and PAL . . . 10
5.5 Inference-time cleanup . . . 11

6 Experiments **11**
6.1 Setup . . . 11
6.2 Baseline model . . . 12
6.3 Architecture comparison (unconditional) . . . 13
6.4 Inference and backward-pass cost . . . 15
6.5 Constraint satisfaction of the Parallel transformer . . . 16
6.6 Latent smoothness . . . 17

7 Limitations and Future Work **19**

8 Conclusion **20**

Appendix A Architecture and training hyperparameters **21**

Appendix B Architecture comparison on converged seeds **23**

1 Introduction

Generative models for spatial layouts have improved quickly in applications like graphic design [1], indoor scene synthesis [2], and floor-plan generation [3]. These systems are typically trained and evaluated on curated datasets of human-designed examples, often alongside geometric or realism-based metrics. Component placement in engineering (for UAVs for example) tackles a different kind of problem. Unlike scene-layout generation tasks, engineering placement primarily emphasizes feasibility and constraint satisfaction. Earlier engineering methods [4] tackled this directly with simulated annealing and genetic algorithms, but they output one optimized layout at a time, not a generator that can produce many.

Recent work in differentiable engineering design starts to bridge the gap between classical engineering optimization and generative modeling. The GLUE framework [5] combines frozen subsystem generators for a full UAV and backpropagates gradients through each one. That setup puts strict demands on every generator. The latent space has to stay smooth and the model actually needs to depend on the latent. Inside GLUE, the internals subsystem is a natural target for improvement. It generates axis-aligned boxes as placeholder for components such as batteries, electronics, and payload components. Architecturally, it is a simple model. However, the spatial placement problem is substantially more difficult. Every placement affects every other one through overlap and packing constraints. The current model uses a non-attentive autoregressive MLP, and performance drops as the number of boxes increases.

This work investigates which architectures are best suited for differentiable component placement within GLUE. We compare three models under the same losses and evaluation setup: a non-attentive MLP baseline, an autoregressive transformer using DiT blocks [6], and a parallel decoder inspired by transformers such as DETR [7]. We also extend the parallel model to support signed pairwise adjacency constraints through a constraint encoder.

For now the setting is deliberately limited to axis-aligned boxes, simple geometry, and a small number of components. The architectures we study here are meant to lay the groundwork for a more general component placement model, one that can be trained efficiently without requiring datasets of pre-optimized layouts.

1.1 Contributions

Our contributions are as follows:

1. A generative placement model that produces 3D box configurations conditioned on relational constraints such as adjacency graphs.

2. A latent-smoothness analysis verifying that the placement model's gradient with respect to z is well-behaved (smooth) end-to-end.

3. A systematic comparison of differentiable architectures for 3D box layout: MLP, AR transformer, and parallel decoder.

2 Related Work

2.1 Layout generation

Generative models for spatial layouts span several representations and architectures. We focus on three papers that most directly influenced our design choices.

LayoutDM [1] formulates conditional 2D graphic layout generation as a denoising diffusion problem with a transformer operating on element bounding boxes and attributes. One result matters most for us: transformer architectures alone can produce high-quality, constraint-aware layouts. That observation motivated our move away from the non-attentive MLP baseline.

We did not pursue the diffusion formulation itself, though not because it is unsuited to the task. Diffusion and flow-matching models produce high-quality, diverse samples, and modern few-step samplers keep their cost modest. The obstacle in our case is data: such models are usually trained to match a dataset of example layouts, whereas the GLUE internals subsystem is trained fully data-free, optimizing geometric and relational constraint losses directly. Using diffusion here would first require a corpus of high-quality feasible layouts to learn from, which we chose to avoid in order to keep the constraint-driven setup. We therefore study architectures that train directly under these losses, and leave a diffusion-based variant as promising future work.

ATISS [2] generates 3D indoor scenes by predicting object bounding boxes with a transformer trained on 3D-FRONT. The model treats scenes as unordered sets rather than sequences, making the generation process permutation-invariant. Our AR-Transformer (Section 4.2) is closest to ATISS in spirit. Both models emit 3D boxes conditioned on previously generated ones. The training setup is different, though. ATISS learns from curated human-designed scenes, whereas our model is trained fully data-free under geometric losses and latent conditioning to match the GLUE interface. The training instability we observe for the autoregressive model at $N = 10$ (Section 6) also pushed us toward a Parallel decoder architecture similar to the one used in ATISS, which removes sequential generation entirely.

House-GAN [3] generates 2D floor plans as axis-aligned room boxes conditioned on a bubble diagram describing room adjacencies. Constraints are encoded directly into the relational message passing structure of the generator and discriminator. Our conditional model starts from a similar input and output format: a pairwise adjacency tensor and a set of axis-aligned boxes. The conditioning mechanism is different. Instead of wiring the graph into the network topology, we map the signed adjacency tensor through a small MLP, then inject the result through adaLN-Zero conditioning (Section 4.4). That keeps unconditional models unchanged at initialization and lets the conditional model extend it directly.

2.2 Differentiable design pipelines

This work builds directly on GLUE [5], which coordinates frozen subsystem generators for wings, airfoils, fuselages, and internals through a shared system-level latent ζ. Gradients are backpropagated through a differentiable geometry layer into each submodel's latent space. That setup imposes two practical requirements on every subsystem generator: (i) the full mapping

must remain differentiable, and (ii) the latent-to-output map $z \mapsto x$ needs to stay smooth,[1] so small changes in ζ lead to predictable geometric updates during coordination.

We focus on the internals subsystem. In the original GLUE implementation, it is a non-attentive autoregressive MLP that emits axis-aligned bounding boxes representing batteries, electronics, and other internal components. Training happens online against differentiable overlap, compactness, and target-volume losses. We keep the same loss formulation and data-free training setup, but replace the architecture (Section 4) and extend it to support signed relational constraints (Section 4.4). The interface itself stays unchanged, so the new model can drop directly into the existing orchestrator.

3 Problem Formulation

We aim to generate layouts of N axis-aligned 3D boxes that respect geometric and possibly relational constraints. The number of boxes varies up to $N_{\max} = 10$, and each box is assigned a prescribed target volume.

Each box $i \in \{1, \dots, N\}$ is parameterized as

$$b_i = (l_i, w_i, h_i, x_i, y_i, z_i) \in \mathbb{R}^6, \tag{1}$$

where (l_i, w_i, h_i) denote the box dimensions and (x_i, y_i, z_i) its center position. Dimensions are restricted to the predefined physical ranges $l_i \in [0.1, 1.5]$, $w_i \in [0.1, 0.5]$, $h_i \in [0.1, 0.5]$ (in metres), and we impose the symmetry constraint $y_i = 0$ for all i, reflecting the mirror symmetry of the target engineering setting. The latent code is $\mathbf{z} \in \mathbb{R}^4$, sampled from $\mathcal{N}(\mathbf{0}, \mathbf{I})$.

Volume matching. Each box has a target volume $v_i \in [v_{\min}, v_{\max}]$, and the generated dimensions should satisfy $l_i w_i h_i \approx v_i$.

Overlap constraint. The pairwise overlap volume between two boxes,

$$V_{\text{ovl}}(b_i, b_j) = \prod_{k \in \{x,y,z\}} \max\Big(0,\ \tfrac{d_{i,k}+d_{j,k}}{2} - |c_{i,k} - c_{j,k}|\Big) \tag{2}$$

must vanish for all $i \neq j$, where $c_{i,k}$ and $d_{i,k}$ denote the center coordinate and extent of box i along axis k, respectively.

Relational constraints. Users may optionally specify qualitative spatial relations via a signed target adjacency tensor $T \in \{-s, 0, +s\}^{N \times N \times 3}$. The sign of a nonzero entry encodes the desired direction of relative placement between two boxes along an axis, and its magnitude s, with $0 < s < 1$, sets the target separation strength (we use $s = 0.3$ for now, though it can be changed). The realized relations are measured by a differentiable smooth-sign adjacency

$$\mathcal{A}(\mathbf{B})_{ijk} = \tanh\big(\alpha(c_{i,k} - c_{j,k})\big), \tag{3}$$

[1] We use *smooth* informally to mean that $z \mapsto x$ has no sharp jumps and a bounded Jacobian $\partial x / \partial z$ (a moderate Lipschitz constant), rather than in the strict C^∞ sense. We assess it qualitatively through the latent sweeps in Section 6.6.

and the layout should satisfy $\mathcal{A}(\mathbf{B})_{ijk} \approx T_{ijk}$ on entries where $T_{ijk} \neq 0$.

Generator. We seek a differentiable generator $G_\theta : (\mathbf{v}, \mathbf{z}, T) \mapsto \mathbf{B}$ taking target volumes $\mathbf{v}$, a latent code $\mathbf{z}$, and an optional relation tensor T, that satisfies the constraints above and remains differentiable with respect to $\mathbf{z}$ to support gradient-based outer-loop optimization. In the unconditional setting, T is omitted.

Beyond these constraints, we also care about softer qualities of the generated layouts, namely compact packing, diversity across samples, and a latent code that the model genuinely uses and maps interpretably to geometry. These are not imposed as hard constraints but encouraged through the training objectives in Section 5.

4 Architectures

4.1 MLP baseline

The simplest architecture generates boxes autoregressively without attention (Figure 1). The model consists of two MLPs: an embedder ϕ (four LeakyReLU layers, widths $256, 256, 256, 1024$) that maps each generated box to a 1024-dimensional feature, and a generator ψ (four LeakyReLU hidden layers of widths $128, 64, 64, 64$) that produces the next box from the accumulated state. At step i:

$$b_i = \psi(e_{i-1},\, \mathbf{z},\, N,\, v_i), \tag{4}$$

with $e_{i-1} = \sum_{j<i} \phi(b_j)$ initialized to $e_0 = \mathbf{0}$ and updated by $e_i = e_{i-1} + \phi(b_i)$.
The sum-based accumulator is order-invariant in the previously placed boxes. The sequential information enters only through the per-step volume target.

The six-dimensional output of ψ is split: dimensions pass through a sigmoid and are rescaled to their physical ranges (Section 3), while positions are returned unchanged except for the symmetry $y_i := 0$. An auxiliary head, trained jointly through the MI loss (Section 5), reconstructs the latent code $\mathbf{z}$ from the generated positions and the running embedding.

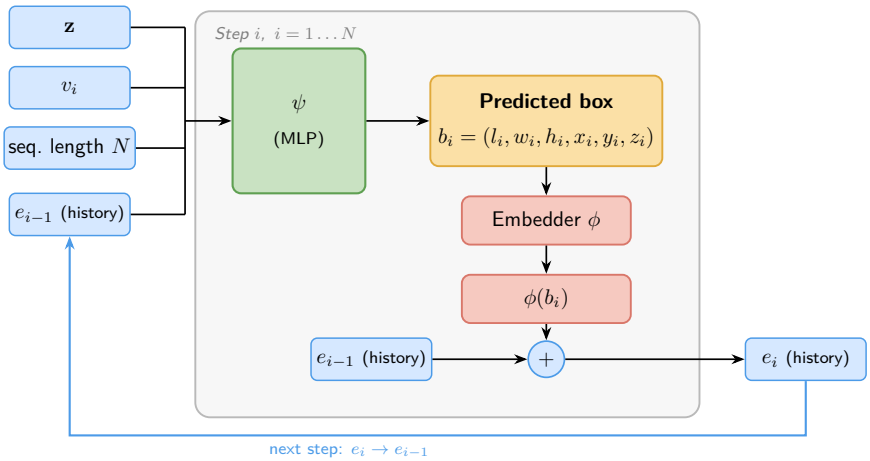


Figure 1: MLP baseline: a single autoregressive step. Previously placed boxes enter only through the summed history embedding.

4.2 Auto-regressive transformer

Boxes are generated one at a time, each conditioned on the tokens already emitted (Figure 2). The model uses $L = 4$ DiT-style transformer blocks [6] of width $D = 256$ with 8 attention heads (32 dimensions per head) and an MLP expansion ratio of 2 (hidden width 512, GELU activation). Attention uses a causal mask so token i attends only to tokens $\leq i$. The blocks use adaLN-Zero modulation [6] (Figure 3): the latent code $\mathbf{z}$ is mapped to a conditioning vector $\mathbf{c} = \mathrm{MLP}_z(\mathbf{z}) \in \mathbb{R}^D$ (a two-layer SiLU MLP), which predicts a per-channel scale, shift, and gate for each sub-block, modulating every block's normalization and residual gates rather than entering the token sequence directly. The modulation projection is zero-initialized, so each block reduces to the identity at the start of training. The exact operations are given in Appendix A (Equations 14–16).

Target volumes and the six box coordinates are encoded with sinusoidal Fourier features before projection to the model dimension. A scalar a is mapped to $[\sin(a\boldsymbol{\sigma}), \cos(a\boldsymbol{\sigma})]$ with frequencies $\sigma_k = 2\pi\, v^{-2k/P}$, $k = 0, \ldots, P/2 - 1$ and base $v = 0.1$. We use $P = 64$ for the volume target and $P = 42$ for each of the six box coordinates, after which a linear layer projects to D. Learned positional embeddings index sequence slots. Per-box output parameterization, the auxiliary head, and all layer widths are specified in Appendix A.

At step i, a query token derived from v_i (its Fourier encoding) is appended to the current sequence. The model predicts the next box from the final hidden state h_i concatenated with v_i through a two-layer LeakyReLU head:

$$\hat{b}_i = \mathrm{Head}(h_i \,\|\, v_i) \in \mathbb{R}^6. \tag{5}$$

The raw output is mapped to a valid box by passing the three dimension channels through a sigmoid rescaled to their physical ranges (Section 3), e.g. $l_i = 0.1 + 1.4\,\sigma(\hat{b}_{i,1})$, while the position channels x_i, z_i are taken directly and $y_i = 0$. The resulting box is encoded through the same Fourier pipeline and appended as a slot token for the next step. An auxiliary head (a small LeakyReLU MLP, Appendix A) reconstructs $\mathbf{z}$ from (x_i, y_i, z_i), h_i and the sequence length for the MI loss (Section 5).

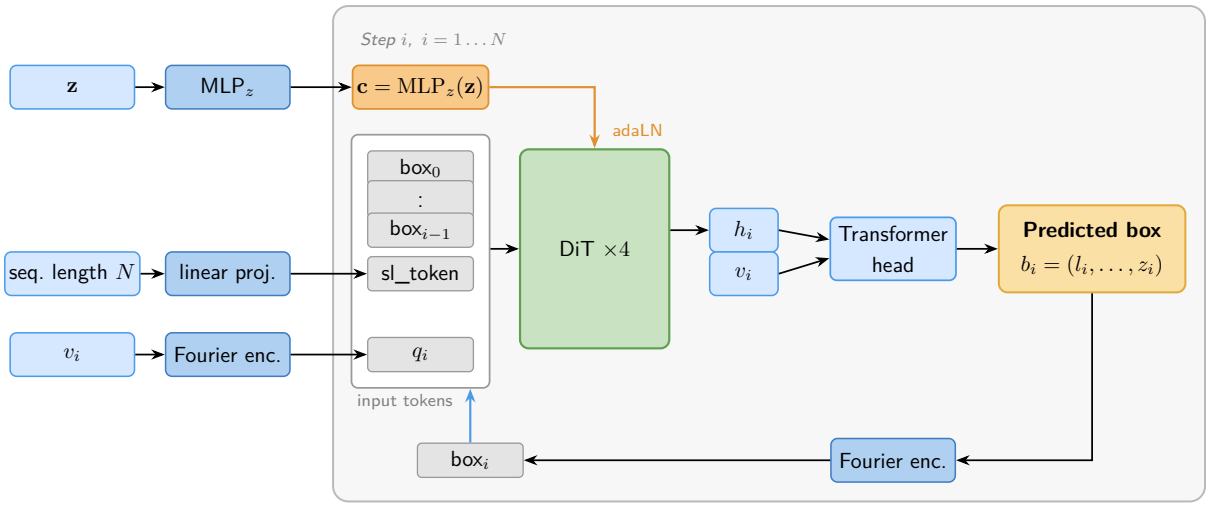


Figure 2: Autoregressive transformer: one box is emitted per step, attending over all previously emitted box tokens.

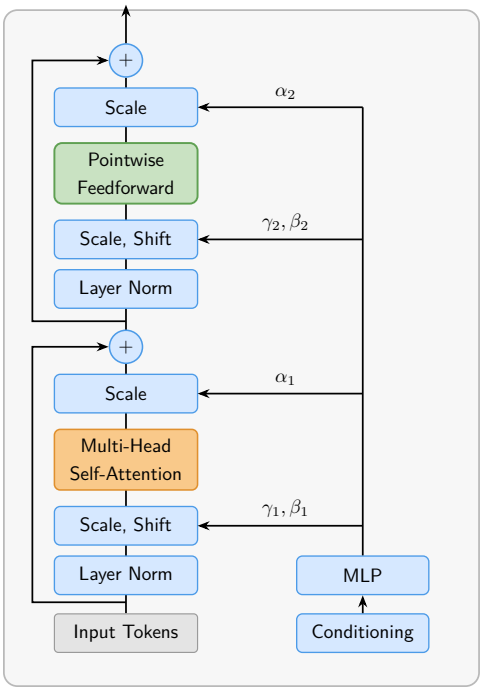


Figure 3: A single DiT block with adaLN-Zero conditioning, shared by both transformer architectures. Adapted from Peebles and Saxena [6].

4.3 Parallel decoder transformer

All boxes are generated in a single forward pass (Figure 4). Inspired by the DETR design [7], the decoder operates on a set of N learned queries (one per box) that self-attend to each other and cross-attend to a small context, with all output positions produced simultaneously rather than autoregressively.

Queries are constructed by adding Fourier-encoded target volumes to per-slot learned embeddings $q_i = \text{Vol}(v_i) + e_i$, where $e_i \in \mathbb{R}^D$ is a learned embedding indexed by slot position. The context contains the projected latent code the seq-length token. The geometry of placed boxes

does not appear in it.

The decoder stacks $L = 4$ pairs of DiT blocks [6] at the same width and head count as the autoregressive model ($D = 256$, 8 heads, MLP ratio 2): at each layer, queries first self-attend (allowing boxes to coordinate placement decisions), then cross-attend to the context. Both block types use adaLN-Zero modulation conditioned on $\mathbf{c}$, identically to the autoregressive transformer. The cross-attention block applies the same shift/scale/gate modulation to a query-to-context attention instead of self-attention. Self-attention here is unmasked, since all boxes are produced jointly.

All boxes are predicted from the final query hidden states in one operation:

$$b_i = \mathrm{Head}(h_i \,\|\, v_i), \qquad i = 1, \ldots, N. \tag{6}$$

Output scaling and the symmetry $y_i = 0$ follow Section 3. An auxiliary head reconstructs $\mathbf{z}$ from each generated box for the MI loss (Section 5).

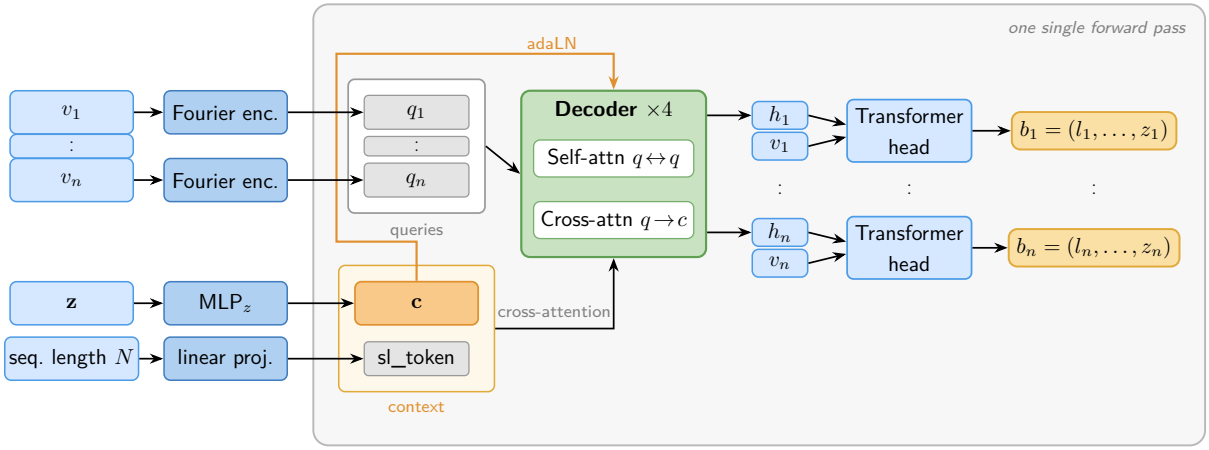


Figure 4: Parallel decoder transformer: all N boxes are produced in a single forward pass, with queries cross-attending to a context of the latent and sequence-length tokens.

4.4 Conditional variants

We extend the parallel decoder to support relational constraints with a small modification (Figure 5). A constraint encoder Φ maps the signed adjacency target T to a vector that is added to the conditioning vector $\mathbf{c}$ produced by the latent projection,

$$\mathbf{c} \;\leftarrow\; \mathbf{c} + \Phi(T). \tag{7}$$

The resulting $\mathbf{c}$ is then passed through the same adaLN-Zero conditioning pathway as before. If no target is provided, the model falls back to the unconditional version. The same mechanism also works for the autoregressive transformer because both models share the same conditioning structure, though in practice we only evaluate it for the parallel decoder.

The encoder Φ is implemented as a small MLP operating on a flattened, zero-padded target tensor. We first pad the input to size $N_{\max} \times N_{\max} \times 3$, reshape it into a vector of dimension $3N_{\max}^2$, then project it through two SiLU-activated hidden layers of width $D = 256$ into the model dimension. The output projection is zero-initialized, so initially $\Phi(T) = \mathbf{0}$ and the conditional

model behaves exactly like the unconditional one. That makes early training much easier, since the transformer blocks do not need to adapt immediately to a randomly initialized constraint signal.

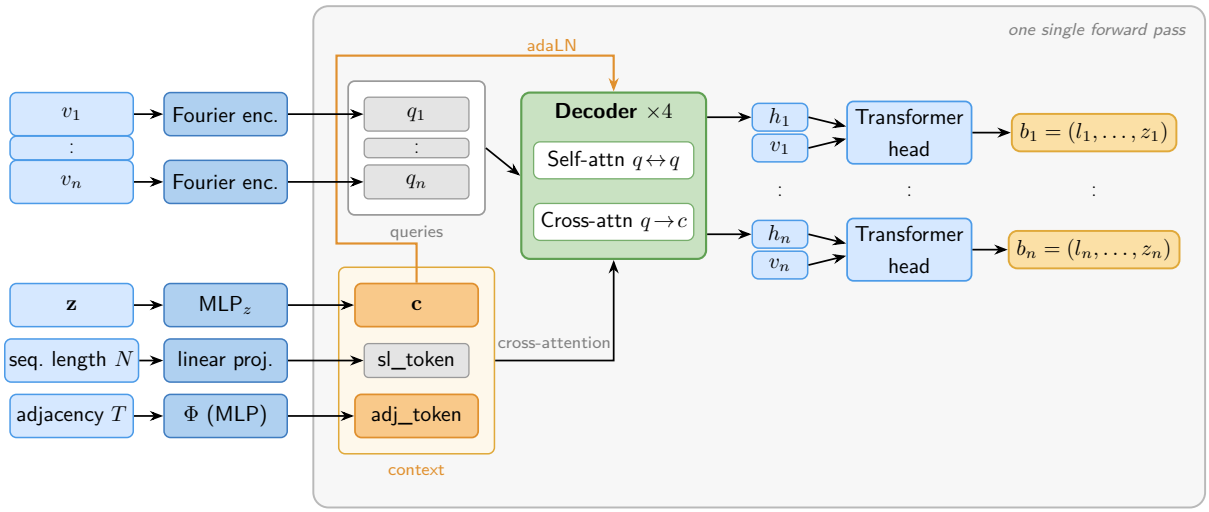


Figure 5: Conditional parallel decoder: the encoded adjacency target T is added to the context as an extra token, otherwise identical to Figure 4.

5 Training Objectives

5.1 Geometric losses

Three losses enforce the constraints from Section 3 in soft form.
The volume residual penalizes deviations from each target:

$$\mathcal{L}_{\text{vol}} = \left\| \frac{l_i w_i h_i - v_i}{v_i} \right\|_2, \tag{8}$$

normalized so that errors are scale-invariant.
The overlap loss is the Frobenius norm of the pairwise overlap matrix $O_{ij} = V_{\text{ovl}}(b_i, b_j)$, divided by the ℓ_2 norm of the target volume vector:

$$\mathcal{L}_{\text{ovl}} = \frac{\|O\|_F}{\|\mathbf{v}\|_2}. \tag{9}$$

The compactness loss is the side-view bounding-box area of the layout, normalized by the total target volume raised to the 2/3 power so the ratio is dimensionless:

$$\mathcal{L}_{\text{comp}} = \frac{A_{\text{side}}(\mathbf{b})}{(\sum_i v_i)^{2/3}}. \tag{10}$$

5.2 Diversity: Aspect Ratios DPP and Layout Diversity

Two determinantal point process (DPP) losses encourage varied layouts across the batch rather than collapse to a single mode. Both build a Gaussian kernel over per-example feature vectors f_i,

$$K_{ij} = \exp\left(-\frac{\|f_i - f_j\|^2}{2\sigma^2}\right), \tag{11}$$

and minimize $-\frac{1}{B}\log\det K$, which is small when the f_i remain distinct and large when they collapse to similar solutions.

The AR-DPP loss (aspect-ratio DPP) uses the log aspect ratios $f_i = \log(w_i/l_i,\, h_i/l_i)$ of the generated boxes, encouraging diversity in box *shape*.

The Layout Diversity (LDV) loss uses the signed adjacency matrices $\mathcal{A}(\mathbf{B})$ along the x and z axes, encouraging diversity in the relative *ordering* of boxes.

5.3 InfoNCE

To prevent the model from ignoring the latent code, an auxiliary head predicts $\hat{\mathbf{z}}$ from each generated box (Section 4). A contrastive InfoNCE loss [8] ties the prediction to the sampled latent:

$$\mathcal{L}_{\text{MI}} = -\frac{1}{B}\sum_{i=1}^{B}\log\frac{\exp(\tilde{\mathbf{z}}_i^\top\tilde{\hat{\mathbf{z}}}_i/\tau)}{\sum_{j=1}^{B}\exp(\tilde{\mathbf{z}}_i^\top\tilde{\hat{\mathbf{z}}}_j/\tau)}, \tag{12}$$

where $\tilde{\cdot}$ denotes ℓ_2 normalization, $\tau = 0.1$ is a temperature, and the negatives are drawn from other examples in the same batch. Minimizing $\mathcal{L}_{\text{MI}}$ maximizes a lower bound on the mutual information between $\mathbf{z}$ and the model's output, forcing the generator to produce layouts whose latent identity is recoverable from the boxes themselves.

5.4 Weighted-sum and PAL

The default training regime is a straightforward weighted sum of all loss terms:

$$\mathcal{L} = \lambda_{\text{vol}}\mathcal{L}_{\text{vol}} + \lambda_{\text{ovl}}\mathcal{L}_{\text{ovl}} + \lambda_{\text{comp}}\mathcal{L}_{\text{comp}} + \lambda_{\text{AR}}\mathcal{L}_{\text{AR-DPP}} + \lambda_{\text{LDV}}\mathcal{L}_{\text{LDV}} + \lambda_{\text{MI}}\mathcal{L}_{\text{MI}} + \lambda_{\text{cond}}\mathcal{L}_{\text{cond}}, \tag{13}$$

where $\mathcal{L}_{\text{cond}}$ is a masked MSE between the realized and target adjacency on the constrained entries (used only by the conditional model). The constraint and objective weights follow a ramp schedule: early epochs use lower weights to let the model first learn coarse geometry, with weights linearly ramped to their final values between epochs 500 and 1500.

We additionally explored a projection-based training scheme (PAL)[2] that alternates a Levenberg–Marquardt step toward the feasible manifold with an exponential-moving-average update of Lagrangian multipliers on the constraint residuals. We found that PAL improved feasibility and quality and therefore opted for it for the unconditional models.
The hyperparameters for both training methods are listed in Appendix A (Table 7).

[2]PAL is based on unpublished work by Tim Aebersold and, as of 5 June 2026, has not yet been published.

5.5 Inference-time cleanup

The raw generator output is not always feasible. Boxes can overlap or miss their target volumes. During inference we optionally run a *cleanup* step that pushes the layout toward the feasible set. The procedure remains differentiable, so gradients can still flow back to the latent code and through the full GLUE pipeline.

Cleanup is implemented as a projection step using Levenberg–Marquardt with Yamashita–Fukushima damping. We iterate until the constraint residuals fall below a tolerance or until a fixed iteration budget is exhausted. The constraint function includes both the normalized volume residual $(l_i w_i h_i - v_i)/v_i$ and the pairwise overlap volume, so the optimizer adjusts box dimensions and positions together. Because the volume constraint is solved to convergence, volume accuracy after cleanup becomes effectively exact by construction. Reported VolAcc values with cleanup therefore mostly reflect whether the projector converged, rather than the quality of the raw network output.

Throughout the paper, *cleanup off* refers to the direct model output, while *cleanup on* refers to the projected layout after this refinement step.

6 Experiments

6.1 Setup

Training. Unconditional models are trained with the projection-based PAL scheme (Section 5). The weighted-sum ramp was originally tuned for the $N_{\max} = 3$ setting and did not transfer cleanly to $N_{\max} = 10$, where overlap infeasibility became much harder to control. This motivated the use of the projector for the unconditional experiments. The conditional model instead uses the weighted-sum objective. The projector still produced clean, low-overlap layouts here, but on visual inspection the weighted-sum ramp satisfied the requested relations more reliably. We did not investigate why in detail. The most likely reason is that the projector's constraint function only enforces the geometric residuals (volume and overlap) and is blind to the adjacency targets, so it can push boxes toward feasibility in ways that break the requested relations. A constraint-aware projector that also accounts for the adjacency targets would be worth trying, since it gave the cleaner geometry of the two schemes.

All models are trained for 2500 epochs with Adam at learning rate 5×10^{-4}, batch size 256, and $N_{\max} = 10$. Target volumes are sampled uniformly from $[0.005, 0.1]$. Each unconditional architecture is trained across 10 random seeds with shared hyperparameters. The conditional Parallel model is likewise trained across 10 seeds, using the weighted-sum ramp objective (Section 5) rather than the projector.

Evaluation protocol. Unconditional models are evaluated on 100 random latent samples per seed, with mean ± standard deviation reported across the 10 seeds. The conditional model is evaluated on 100 random latent samples per seed across its 10 training seeds. Unconditional models are evaluated at two sequence lengths, $N \in \{3, 10\}$, and the conditional model at $N \in \{3, 10\}$ as well. Each model is evaluated in two settings: direct generator output (*cleanup*

off) and output after the projection step from Section 5.5 (*cleanup on*). This separates the behavior of the learned model from the effect of the refinement step.

Metrics. We report five primary metrics:

- *Overlap*: mean total pairwise box overlap volume (lower is better).
- *Vol. Acc.*: $1 - \mathbb{E}\big[|l_i w_i h_i - v_i|/v_i\big]$ (higher is better).
- *Compactness*: fill ratio $\sum_i v_i / V_{\text{bbox}}$ (higher is denser packing).
- *LDV* and *MI*: as defined in Section 5, lower is better for both.

For the conditional model we additionally report *Satisfaction*: the fraction of specified pairwise adjacency relations whose intended direction is realized in the generated layout (computed as the fraction of entries with $T_{ijk} \neq 0$ for which $\operatorname{sign} \mathcal{A}(\mathbf{B})_{ijk} = \operatorname{sign} T_{ijk}$).

Conditional density sweep. For the conditional model we further sweep the constraint density $\rho \in \{0.1, 0.2, 0.3, 0.5, 0.7, 0.9\}$ at $N = 10$, where ρ controls the probability that each unordered pair receives a relation in the sampled target adjacency matrix. This characterises how satisfaction trades off against geometric validity as the constraint problem becomes harder.

Latent smoothness. For each architecture we additionally produce qualitative smoothness sweeps: holding volumes and all but one latent coordinate fixed, we sweep that coordinate across $[-3.5, 3.5]$ in 50 points and plot the resulting box-center positions, both with and without inference-time cleanup. This visualises whether $\mathbf{z} \mapsto \mathbf{x}$ varies smoothly.

Compute. All training and evaluation jobs were dispatched as SLURM arrays on the ETH Euler cluster, with one NVIDIA RTX 4090 GPU per job. Training each model takes approximately 44 minutes for the autoregressive models and 16 minutes for the parallel.

6.2 Baseline model

seq_len	cleanup	Overlap ↓	VolAcc ↑	Compactness ↑	LDV ↓	MI ↓
3	off	0.021 ± 0.017	**0.80 ± 0.27**	0.50 ± 0.08	21.1 ± 2.2	3.6 ± 1.0
3	on	**0.015 ± 0.014**	**0.80 ± 0.27**	0.49 ± 0.07	21.1 ± 2.2	3.6 ± 1.0
10	off	0.389 ± 0.262	0.78 ± 0.28	**0.62 ± 0.17**	**3.5 ± 7.6**	**3.4 ± 1.1**
10	on	0.377 ± 0.256	0.78 ± 0.28	0.61 ± 0.17	**3.5 ± 7.7**	**3.4 ± 1.1**

Table 1: Original MLP baseline (ramp training schedule), as used in the initial GLUE internals model. Mean ± std across 10 seeds. Arrows indicate whether higher (↑) or lower (↓) is better; the best value per column is shown in bold.

Table 1 reports the original GLUE internals model, the non-attentive MLP of Section 4.1 trained with the ramp schedule, and is the baseline we set out to beat. Overlap grows from 0.021 at $N = 3$ to 0.39 at $N = 10$ (about 19×), leaving most $N = 10$ layouts infeasible, while volume

accuracy stays around 0.78–0.80. Cleanup changes little at $N = 10$: overlap drops only from 0.39 to 0.38 and volume accuracy stays at 0.78, so the raw layouts are too infeasible for the projector to converge. The $N = 10$ figures are partly a tuning issue rather than a fixed architectural limit, since the ramp weighting was set for $N_{\max} = 3$ and does not transfer to $N_{\max} = 10$ (Section 6.1). The architectures below address these problems, and both attention models reach far lower overlap and higher volume accuracy at $N = 10$.

6.3 Architecture comparison (unconditional)

model	seq_len	Cleanup	Overlap ↓	VolAcc ↑	Compactness ↑	LDV ↓	MI ↓
MLP	3	off	0.009 ± 0.005	0.38 ± 0.01	0.41 ± 0.08	22.6 ± 0.5	3.8 ± 0.7
MLP	3	on	0.004 ± 0.001	**1.00 ± 0.00**	0.50 ± 0.04	22.4 ± 0.5	3.8 ± 0.7
MLP	10	off	0.094 ± 0.040	0.35 ± 0.01	0.41 ± 0.08	0.91 ± 0.52	4.1 ± 1.4
MLP	10	on	0.024 ± 0.003	**1.00 ± 0.00**	0.49 ± 0.02	0.31 ± 0.16	4.1 ± 1.4
AR-Transformer	3	off	0.010 ± 0.013	0.87 ± 0.24	0.55 ± 0.19	21.2 ± 0.7	3.0 ± 3.3
AR-Transformer	3	on	**0.003 ± 0.001**	**1.00 ± 0.00**	0.53 ± 0.19	21.2 ± 0.7	3.0 ± 3.3
AR-Transformer	10	off	0.124 ± 0.150	0.87 ± 0.25	0.61 ± 0.22	1.09 ± 3.00	3.2 ± 3.3
AR-Transformer	10	on	0.033 ± 0.015	**1.00 ± 0.00**	0.58 ± 0.22	0.94 ± 2.54	3.2 ± 3.3
Parallel	3	off	0.008 ± 0.003	0.98 ± 0.01	0.60 ± 0.05	21.1 ± 0.4	**1.4 ± 0.1**
Parallel	3	on	**0.003 ± 0.001**	**1.00 ± 0.00**	0.58 ± 0.05	21.1 ± 0.4	**1.4 ± 0.1**
Parallel	10	off	0.102 ± 0.028	0.98 ± 0.01	**0.64 ± 0.04**	0.14 ± 0.13	**1.4 ± 0.1**
Parallel	10	on	0.031 ± 0.011	**1.00 ± 0.00**	0.60 ± 0.03	**0.13 ± 0.12**	**1.4 ± 0.1**

Table 2: Architecture comparison at $N \in \{3, 10\}$, with and without inference-time Cleanup. Mean ± std across 10 training seeds. Best value per column in bold (higher better for ↑, lower for ↓).

Attention vs. the MLP. The largest gap in Table 2 is between the MLP and the attention-based models. At $N = 10$, the MLP reaches raw VolAcc 0.35, against 0.98 for the Parallel decoder (and 0.99 on the AR seeds that converge, see below). MI is also much worse for the MLP: 4.1 versus 1.4 for the Parallel decoder. The same pattern appears in LDV, where the MLP reaches 0.91 and the Parallel decoder stays at 0.14. Raw overlap is the only metric where the MLP stays competitive (0.094 versus 0.10 for the Parallel decoder).

One possible explanation is the different representation of previously generated boxes. Attention keeps a separate token per box and models their pairwise interactions directly through $Q_i K_j^\top$, whereas the MLP collapses the history into a single summed embedding.

AR seed divergence. The AR-Transformer shows much larger seed variance than the Parallel decoder on every quantitative metric. At $N = 10$, the AR standard deviation is 39× larger for VolAcc, 25× for MI, and 6× for compactness. The same pattern appears at $N = 3$. This variance is not a smooth spread across seeds: it is driven almost entirely by two of the ten seeds whose training diverged outright. On those two runs the box-center coordinates grew without bound (per-sample position spread of 40 and 6100 versus ≈ 0.2 on the stable runs), dragging VolAcc down to 0.24–0.54 and MI up to 9–10. The remaining eight seeds are tightly clustered and competitive with the Parallel decoder (VolAcc 0.989 ± 0.005, MI 1.57 ± 0.50, overlap 0.082 ± 0.033

at $N = 10$). Table 8 in Appendix B repeats the full comparison with these eight converged AR seeds. The training logs trace the cause of the divergence to the projector ramp: the constraint multiplier λ_c grows almost linearly toward its ceiling of 25 on an identical schedule for every seed, so the gradient through the $\lambda_c \cdot |\text{residual}|$ constraint term grows with it. Both diverged runs are stable for 1700–2200 epochs and then spike by five orders of magnitude within one logging interval, once λ_c has grown large enough ($\approx$ 18–22) that a single unlucky batch pushes the weights out of the feasible basin and the residual-driven gradient compounds. Gradient-norm clipping bounds the step size but not its direction, so it does not arrest the feedback. A slower or feasibility-gated λ_c ramp could be a fix, though it would probably need further tuning.

Raw overlap. Averaged over all ten seeds at $N = 10$ before cleanup, the AR model reaches overlap 0.12 and the Parallel decoder 0.10. After cleanup they move to 0.033 and 0.031. The AR average is inflated by the two diverged seeds: on the eight converged AR runs raw overlap is 0.082 ± 0.033, marginally lower than Parallel. Any such edge would plausibly come from the AR model's direct access to previously generated boxes: at step i it attends to $b_{<i}$ and can steer away from occupied regions, whereas the Parallel decoder must coordinate placements through self-attention between queries that do not yet contain the final geometry. In any case the difference is small and disappears after cleanup.

Effect of cleanup. For both transformers, cleanup lowers overlap while also reducing compactness (AR 0.61 $\rightarrow$ 0.58, Parallel 0.64 $\rightarrow$ 0.60). For the MLP, cleanup improves overlap from 0.094 $\rightarrow$ 0.024 (4$\times$) and compactness from 0.41 $\rightarrow$ 0.49. This suggests the transformer outputs already operate near the overlap–compactness tradeoff boundary, where improving one hurts the other. The MLP outputs appear farther from feasibility, which leaves more room for cleanup to improve both metrics at once. Cleanup still helps the transformers, though: only the projection step guarantees true non-overlap, whereas the soft penalty alone leaves them on the overlap–compactness tradeoff.

Parallel's advantage. On the converged AR seeds the two transformers are close on the geometric metrics, and AR even has marginally lower raw overlap at $N = 10$. We would not read the AR instability as a fundamental architectural weakness, since it stems from the projector ramp and is probably tunable. The clearer architectural difference is latent usage: on the smoothness sweeps (Section 6.6) the Parallel decoder responds visibly more to z, plausibly because the AR model can lean on its previously generated boxes instead. The Parallel decoder is also several times cheaper to evaluate and differentiate, since it emits all boxes in one pass rather than N sequential ones (Section 6.4).

ID	z-cond.	Vol. enc.	Vol. head	Layers	Dim. size	Overlap ↓	VolAcc ↑	Compactness ↑	LDV ↓	MI ↓
`Base`	adaLN	Fourier	✓	4	256	**0.102 ± 0.028**	0.981 ± 0.006	0.64 ± 0.04	0.14 ± 0.13	1.45 ± 0.14
`A`	token					0.111 ± 0.031	0.981 ± 0.003	0.61 ± 0.03	0.090 ± 0.040	**1.33 ± 0.05**
`B`	both					0.108 ± 0.029	0.980 ± 0.005	**0.66 ± 0.04**	0.14 ± 0.14	1.56 ± 0.42
`C`		Linear				0.110 ± 0.018	0.916 ± 0.024	0.57 ± 0.03	0.65 ± 0.14	1.53 ± 0.19
`D`			×			0.117 ± 0.025	0.979 ± 0.007	0.65 ± 0.03	0.090 ± 0.050	1.57 ± 0.27
`E`				2	128	0.139 ± 0.021	**0.983 ± 0.003**	0.64 ± 0.03	**0.070 ± 0.060**	1.37 ± 0.05
`F`				6	512	0.106 ± 0.034	**0.983 ± 0.006**	0.65 ± 0.07	0.11 ± 0.04	1.49 ± 0.13

Table 3: Ablations for our Parallel decoder. The `Base` row is the default configuration, and each later row (`A`–`F`) lists only the hyperparameters that change from it, leaving unchanged settings blank. Arrows mark whether higher (↑) or lower (↓) is better, and the best value per column is in bold.

Volume encoding. Replacing the Fourier volume encoder with a linear projection (Table 3, row `C`) drops VolAcc from 0.981 to 0.916 and increases LDV from 0.14 to 0.65. No other single change shifts several metrics by a comparable amount. The other ablations (conditioning route, vol-head concatenation, depth, width) mostly affect one metric at a time, and only modestly. Among the tested design choices, the volume encoding has the largest effect size.

Conditioning route. Within the conditioning ablation (Table 3, rows `Base`, `A`, `B`), each option favors a different metric. adaLN gives the lowest overlap (0.10). Token conditioning gives the lowest MI (1.33). Using both produces the highest compactness (0.66). The gaps are small, but they are consistent across runs. We use adaLN in the main experiments because it gives the best feasibility. Token conditioning may still be preferable when latent recoverability matters more.

Capacity. Reducing the model to 2 layers / 128 dimensions increases overlap from 0.10 to 0.14 (Table 3, row `E`), while the other metrics stay roughly unchanged. Increasing the model to 6 layers / 512 dimensions (row `F`) gives almost no measurable improvement. The 4-layer / 256-dim configuration already sits near the knee of the scaling curve. The remaining feasibility gap does not appear to come from limited model capacity.

6.4 Inference and backward-pass cost

The two decoders differ sharply in runtime, and the gap matters because GLUE repeatedly backpropagates through the generator during outer-loop coordination. We time both transformers on a single RTX 4090 at batch size 100, measuring forward-only (inference) and forward+backward (an overlap loss differentiated to the latent) cost, averaged over 50 timed repetitions and 10 randomly initialized models with CUDA synchronization (Table 4).

	Forward (ms) ↓		Forward+Backward (ms) ↓	
Model	$N = 3$	$N = 10$	$N = 3$	$N = 10$
AR-Transformer	12.85 ± 0.01	42.22 ± 0.05	38.37 ± 0.07	126.06 ± 0.33
Parallel	$\mathbf{7.17 \pm 0.01}$	$\mathbf{8.86 \pm 0.01}$	$\mathbf{21.47 \pm 0.02}$	$\mathbf{23.69 \pm 0.07}$
Speed-up	1.8×	4.8×	1.8×	5.3×

Table 4: Per-call wall-clock cost of the two transformer decoders on a single RTX 4090, at batch size 100 (mean ± std over 10 randomly initialized models, 50 timed repetitions each after warmup, with CUDA synchronization). *Forward* is inference, and *Forward+Backward* additionally backpropagates an overlap loss to the latent, the path GLUE's outer loop exercises. The Parallel decoder produces all boxes in one pass, so its cost is nearly flat in N, whereas the AR-Transformer runs N sequential passes and scales steeply.

Cost and scaling in N. At $N = 10$ the Parallel decoder is 4.8× faster on the forward pass (8.9 vs 42.2 ms) and 5.3× faster on forward+backward (23.7 vs 126.1 ms). The cause is structural: the AR-Transformer runs N sequential forward passes, one per emitted box, so its cost grows steeply with sequence length (3.3× from $N = 3$ to $N = 10$), whereas the Parallel decoder emits all boxes in a single pass and is nearly flat (1.2× over the same range). At $N = 3$ the gap is smaller (1.8×) but already in the Parallel decoder's favour. This advantage compounds in the GLUE setting, where the backward pass is taken many times per design iteration.

6.5 Constraint satisfaction of the Parallel transformer

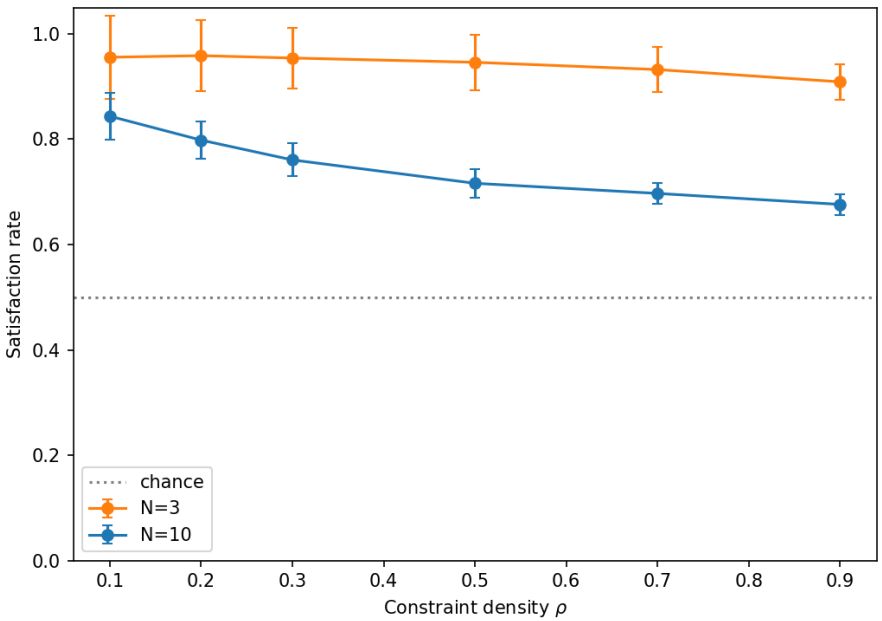


Figure 6: Density vs satisfaction rate (with clean up on)

N	cleanup	Satisfaction ↑	Overlap ↓	VolAcc ↑	Compactness ↑	LDV ↓	MI ↓
3	off	**0.95 ± 0.06**	0.018 ± 0.004	0.993 ± 0.001	0.55 ± 0.03	20.0 ± 0.4	**1.26 ± 0.01**
3	on	**0.95 ± 0.06**	**0.003 ± 0.001**	**1.000 ± 0.000**	0.51 ± 0.02	20.0 ± 0.5	**1.26 ± 0.01**
10	off	0.77 ± 0.03	0.521 ± 0.048	0.993 ± 0.001	**0.82 ± 0.06**	0.037 ± 0.019	1.27 ± 0.02
10	on	0.76 ± 0.03	0.025 ± 0.009	**1.000 ± 0.000**	0.50 ± 0.02	**0.013 ± 0.009**	1.27 ± 0.02

Table 5: Conditional Parallel model evaluated at constraint density $\rho = 0.3$. Mean ± std across 10 training seeds. Arrows indicate whether higher (↑) or lower (↓) is better; the best value per column is shown in bold.

Satisfaction vs. feasibility. Here *feasibility* is geometric, meaning no overlap and correct volumes (Overlap and VolAcc), while *constraint satisfaction* is relational, the fraction of requested adjacency relations realized (the Satisfaction metric). The two can come apart: a perfectly feasible layout can still place two boxes on the wrong sides of each other. At $N = 3$, the model satisfies constraints in roughly 95% of cases (Table 5). Raw overlap is already low (0.018), and cleanup reduces it further to 0.003. At $N = 10$, satisfaction drops to 76%. Raw overlap rises to 0.52, although cleanup brings it back to 0.025 without materially changing the satisfaction rate. This suggests the main difficulty at larger N is not geometric feasibility itself but jointly satisfying the target adjacency relations while placing all boxes consistently. Volume accuracy remains almost perfect at both lengths (0.993 raw and 1.000 after projection), and MI stays stable around 1.26 across all settings, indicating the conditioning encoder does not interfere much with latent usage. The constraint-density sweep (Figure 6) shows the same pattern across ρ: satisfaction degrades gradually as the constraint problem becomes denser, with a noticeably steeper drop at $N = 10$ than at $N = 3$.

6.6 Latent smoothness

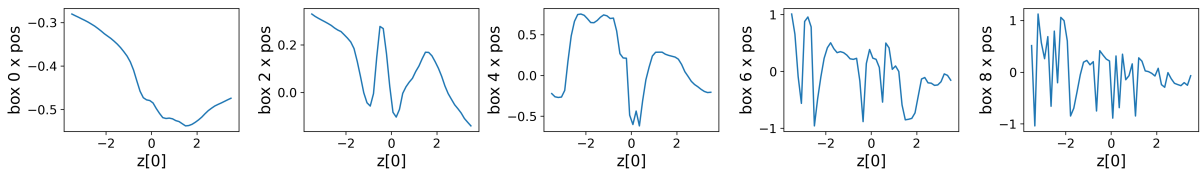


Figure 7: MLP, $N = 10$, cleanup off. Box-center x-position as a function of z0, for boxes 0,2,4,6,8. The rest of the latent coordinates are kept at 0.

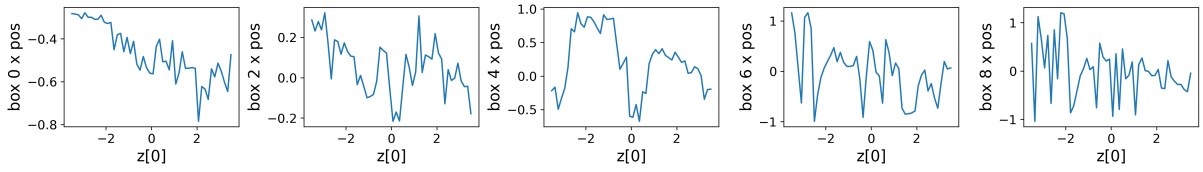


Figure 8: MLP, $N = 10$, cleanup on. Same sweep as Figure 7, with cleanup applied.

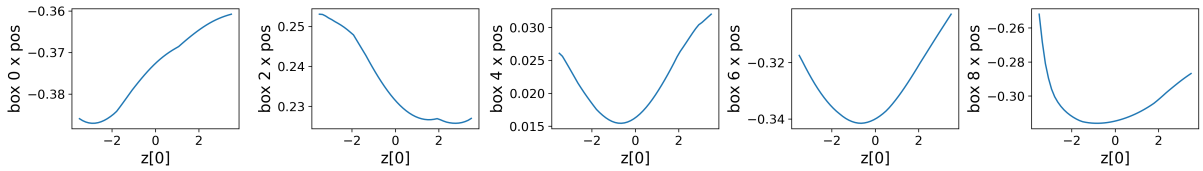


Figure 9: AR-Transformer, $N = 10$, cleanup off. Same sweep as Figure 7.

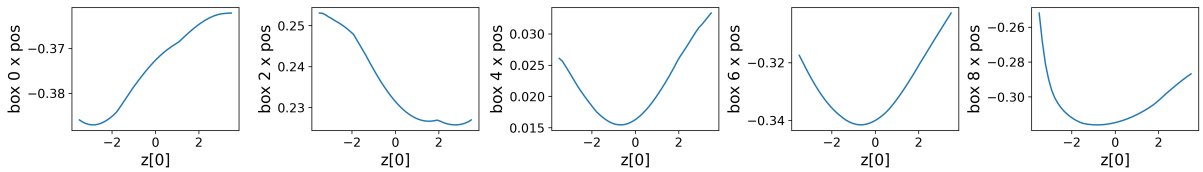


Figure 10: AR-Transformer, $N = 10$, cleanup on. Same sweep as Figure 7, with cleanup applied.

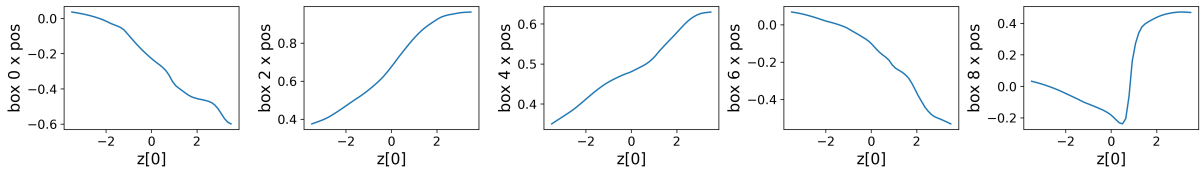


Figure 11: Parallel decoder, $N = 10$, cleanup off. Same sweep as Figure 7.

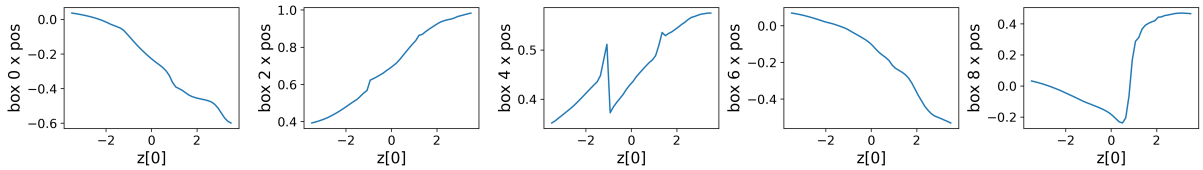


Figure 12: Parallel decoder, $N = 10$, cleanup on. Same sweep as Figure 7, with cleanup applied.

Latent amplitude. Across the swept z range, box positions vary by roughly 0.2–1.0 units for the Parallel decoder, 0.02–0.10 for the AR-Transformer, and about ± 1.0 with visible wiggles for the MLP. The AR model appears to use the latent code much less strongly than the Parallel decoder. We suspect this comes from posterior collapse. Since the AR next-box predictor $p(b_i \mid b_{<i}, v_i, z)$ already receives the previously generated boxes, it can often infer most of the layout structure without relying heavily on z. In practice, z mostly acts as a small perturbation around a default layout. We expect this to be most pronounced at higher box counts, where each step has more placed boxes to read from, which is also the regime our $N = 10$ sweeps cover. The Parallel decoder does not have access to prior boxes, so the latent code has to carry much more of the layout information. We note a tension here: the converged AR seeds report MI scores comparable to the Parallel decoder (1.6 versus 1.4) despite this much weaker latent amplitude, which suggests our InfoNCE-based MI metric may not faithfully capture how strongly the latent drives the geometry, and is worth reconsidering.

This behavior does not seem tied to the specific conditioning mechanism. We tested an AR variant that replaced adaLN conditioning with cross-attention to z, and the latent amplitude

remained similarly small. We emphasise that this weak-latent-use observation concerns the *converged* AR seeds and is separate from the training divergence discussed in Section 6.

MLP smoothness. Both transformer architectures produce smooth $z \mapsto x$ trajectories with relatively low curvature. The MLP behaves differently. Boxes 0–1 remain fairly smooth, but from box 3 onward the trajectories develop large local jumps and sharp wiggles. We think two effects contribute to this behavior. First, the MLP only receives the summed embedding $e_i = \sum_{j<i} \phi(b_j)$ rather than the individual prior boxes. Different layouts can therefore map to similar embeddings, making it difficult for the model to distinguish between them reliably. As z changes, the model can switch abruptly between different interpretations of the current layout state, which shifts the placement of the next box. Attention-based models avoid much of this ambiguity because they retain access to each prior box separately. Second, the MLP outputs are generally farther from feasibility than those of the transformers, especially in overlap and volume accuracy. Cleanup therefore applies larger corrections, which introduces additional discontinuities when the projector moves between different feasible regions. The problem becomes worse for later boxes, where the summed representation is more ambiguous and the raw overlap is higher.

Cleanup kinks. The LM projection step introduces localized C^0 kinks, most clearly in the AR model along z_1 and z_2. We think these occur when the projector switches between nearby feasible regions as z changes. The main downside of cleanup is therefore a loss of smoothness in the $z \mapsto x$ mapping, even though geometric feasibility improves.

7 Limitations and Future Work

Problem scope. This setup is intentionally narrow. Boxes are axis-aligned, dimensions stay within fixed bounds, sequence length is capped at $N_{\max} = 10$, and the symmetry constraint fixes one axis through $y_i = 0$. That keeps the architectural comparison manageable, but many practical issues are missing: rotated geometries, inertia constraints, wiring, and free-form shapes. Extending beyond boxes is difficult because overlap between arbitrary geometries no longer has a simple differentiable form, which the overlap loss in Equation 2 depends on. A smooth differentiable overlap operator for richer shape primitives is probably the main bottleneck for scaling this approach.

Dependence on cleanup. Exact feasibility still depends on the projection step from Section 5.5. The projector remains differentiable, but it introduces localized C^0 discontinuities in the $z \mapsto x$ mapping when optimization switches between nearby feasible regions (Section 6.6). That weakens the smooth outer-loop behavior GLUE relies on. The effect is smaller for the parallel decoder because its raw outputs already lie close to feasibility and need only minor corrections.

Constraint vocabulary. The current interface only supports signed pairwise adjacency along each axis. Real placement problems often require distance bounds, alignment, containment, or soft preferences. The encoder is also fairly limited: it flattens the constraint tensor into an MLP input of size $3N_{\max}^2$, which scales poorly for large N or dense relation graphs. A tokenized

representation with cross-attention, or a graph encoder closer to House-GAN [3], would support richer constraints with little architectural change.

Conditional behaviour at larger N. Constraint satisfaction drops from roughly 95% at $N = 3$ to 76% at $N = 10$ (Section 6.5), while volume accuracy and latent usage stay almost unchanged. The difficulty appears to be satisfying many relations simultaneously while maintaining a coherent layout. A curriculum over constraint density or a separate multiplier scheme for the relational loss would be natural next steps.

Training stability and the projector. The PAL projection scheme worked well for unconditional training but failed in the conditional setting, where we instead used the weighted-sum objective (Section 6.1). We did not fully isolate the cause. One likely issue is that adjacency constraints interact badly with the overlap residual, since reversing a relation often requires boxes to pass through occupied regions before separating again. A single training scheme that handles both settings cleanly would simplify the pipeline considerably.

8 Conclusion

We compared a non-attentive MLP, an autoregressive transformer, and a parallel decoder for differentiable component placement inside GLUE. The clearest result is that attention helps. Both transformers improve substantially over the MLP at $N = 10$ on volume accuracy, latent recoverability, and layout diversity, with the MLP staying competitive only on raw overlap. Among the transformers the difference is one of training stability, latent use, and compute. On the seeds that converge the autoregressive model matches or slightly exceeds the parallel decoder on the geometric metrics, but it diverged on two of ten seeds because of the projector ramp, whereas the parallel decoder trained cleanly on all ten. The parallel decoder also makes the strongest use of the latent and gives the clearest smoothness sweeps: its box positions respond visibly to z across the swept range, while the autoregressive model responds far less despite a comparable MI score, which suggests the latent drives its geometry only weakly. The parallel decoder is also several times cheaper to evaluate and differentiate, since it emits all boxes in one pass rather than N sequential ones (Section 6.4), which matters because GLUE backpropagates through the generator repeatedly during coordination.

For use inside GLUE, where the internals subsystem must train reliably and expose a usable latent, we therefore recommend the parallel decoder. Its main remaining weakness is relational: constraint satisfaction degrades from 95% at $N = 3$ to 76% at $N = 10$ even though geometric feasibility stays high, which makes jointly satisfying many adjacency relations the natural target for future work.

References

[1] Shang Chai, Liansheng Zhuang, and Fengying Yan. “LayoutDM: Transformer-based Diffusion Model for Layout Generation”. In: *Proceedings of the IEEE/CVF Conference on Computer*

Vision and Pattern Recognition (CVPR). 2023, pp. 18349–18358. DOI: 10.48550/arXiv.2305.02567.

[2] Despoina Paschalidou et al. "ATISS: Autoregressive Transformers for Indoor Scene Synthesis". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2021. URL: https://research.nvidia.com/labs/toronto-ai/ATISS/.

[3] Nelson Nauata et al. "House-GAN: Relational Generative Adversarial Networks for Graph-Constrained House Layout Generation". In: *European Conference on Computer Vision (ECCV)*. Springer, 2020, pp. 162–177. DOI: 10.48550/arXiv.2003.06988.

[4] Jonathan Cagan, Kenji Shimada, and Sun Yin. "A survey of computational approaches to three-dimensional layout problems". In: *Computer-Aided Design* 34.8 (2002), pp. 597–611. ISSN: 0010-4485. DOI: 10.1016/S0010-4485(01)00109-9.

[5] Tim Aebersold, Soheyl Massoudi, and Mark D. Fuge. *GLUE: Generative Latent Unification of Expertise-Informed Engineering Models*. arXiv preprint. 2025. DOI: 10.48550/arXiv.2512.19469.

[6] William Peebles and Saining Xie. "Scalable Diffusion Models with Transformers". In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. 2023. DOI: 10.48550/arXiv.2212.09748.

[7] Nicolas Carion et al. "End-to-End Object Detection with Transformers". In: *European Conference on Computer Vision (ECCV)*. Springer, 2020, pp. 213–229. DOI: 10.48550/arXiv.2005.12872.

[8] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. "Representation Learning with Contrastive Predictive Coding". In: *arXiv preprint arXiv:1807.03748* (2018). DOI: 10.48550/arXiv.1807.03748.

Appendix A Architecture and training hyperparameters

Table 6 lists the architectural hyperparameters for all four models, and Table 7 the optimization and loss-weighting settings. The three transformer variants share a common DiT backbone and differ only in how boxes are decoded (causal versus parallel) and in the optional constraint encoder. All adaLN and constraint-encoder output projections are zero-initialized, so every model begins training as a near-identity map perturbed only by the output head.

adaLN-Zero modulation. Each DiT block conditions on $\mathbf{c} = \mathrm{MLP}_z(\mathbf{z})$ through a single linear layer W that predicts a per-channel scale $\boldsymbol{\gamma}$, shift $\boldsymbol{\beta}$, and gate $\boldsymbol{\alpha}$ for each of its two sub-blocks (attention and MLP). Writing LN for a LayerNorm without learnable affine parameters, the block applies

$$(\boldsymbol{\gamma}_1, \boldsymbol{\beta}_1, \boldsymbol{\alpha}_1, \boldsymbol{\gamma}_2, \boldsymbol{\beta}_2, \boldsymbol{\alpha}_2) = W\,\mathrm{SiLU}(\mathbf{c}), \tag{14}$$

$$\mathbf{x} \leftarrow \mathbf{x} + \boldsymbol{\alpha}_1 \odot \mathrm{Attn}\big((1+\boldsymbol{\gamma}_1) \odot \mathrm{LN}(\mathbf{x}) + \boldsymbol{\beta}_1\big), \tag{15}$$

$$\mathbf{x} \leftarrow \mathbf{x} + \boldsymbol{\alpha}_2 \odot \mathrm{MLP}\big((1+\boldsymbol{\gamma}_2) \odot \mathrm{LN}(\mathbf{x}) + \boldsymbol{\beta}_2\big), \tag{16}$$

where $\odot$ is elementwise multiplication. The last layer of W is zero-initialized, so $\boldsymbol{\gamma} = \boldsymbol{\beta} = \boldsymbol{\alpha} = \mathbf{0}$ initially and both residual branches vanish, leaving the identity. The cross-attention block in the Parallel decoder applies the same modulation, with Attn in Equation 15 replaced by query-to-context cross-attention.

Shared transformer backbone (AR-Transformer, Parallel, conditional)	
Model dimension D	256
Transformer blocks L	4
Attention heads	8 (head dim 32)
MLP hidden width / ratio	512 (2×), GELU
Dropout	0.0
LayerNorm	no learnable affine, $\varepsilon = 10^{-6}$
Latent dimension	$\dim \mathbf{z} = 4$
$\mathbf{z}$ projection MLP_z	$4 \to 256 \to 256$, SiLU
Fourier features (volume)	$P = 64$ (32 freqs), base $v = 0.1$
Fourier features (each coord)	$P = 42$ (21 freqs), base $v = 0.1$
Output head	$(D{+}1) \to 256 \to 6$, LeakyReLU
Auxiliary ($\mathbf{z}$) head	$(3{+}D{+}1) \to 64 \to 128 \to 64 \to 4$, LeakyReLU
adaLN modulation	SiLU $\to$ Linear $D \to 6D$, zero-init
Slot / query / pos. embeddings	learned, dimension D
Approx. parameters	AR $\approx$ 3.7M, Parallel $\approx$ 7M
MLP baseline	
Embedder ϕ	$6 \to 256 \to 256 \to 256 \to 1024$, LeakyReLU
Generator ψ	$1030 \to 128 \to 64 \to 64 \to 64 \to 6$, LeakyReLU
Auxiliary head	$1028 \to 64 \to 128 \to 64 \to 4$, LeakyReLU
Approx. parameters	$\approx$ 1M
Constraint encoder Φ (conditional models)	
Architecture	$3N_{\max}^2 \to 256 \to 256 \to 256$, SiLU, zero-init output
Output parameterization	
Dimensions l, w, h	sigmoid rescaled to $[0.1, 1.5]$, $[0.1, 0.5]$, $[0.1, 0.5]$
Positions x, z	unconstrained; $y = 0$ (symmetry)

Table 6: Architecture hyperparameters for all models. The three transformer variants share the backbone above; the conditional models additionally use the constraint encoder Φ. Widths are written as input $\to$ hidden $\to \cdots \to$ output.

Optimization	
Optimizer	Adam
Learning rate	5×10^{-4}
Batch size	256
Epochs	2500
$N_{\max}$	10
Volume sampling	uniform on $[0.005, 0.1]$
Weighted-sum ramp (conditional models)	
Ramp window	epochs $500 \to 1500$ (linear)
λ_{overlap}	$4 \to 20$
λ_{vol}	$5 \to 10$
λ_{comp}	$2 \to 4$
$\lambda_{\text{AR-DPP}}$	$4 \to 1$
λ_{LDV}	$0.5 \to 0.5$
λ_{MI}	$1.0 \to 2.0$
λ_{cond}	$1.0 \to 30.0$
InfoNCE temperature τ	0.1
PAL projector (unconditional models)	
Objective weights	comp. 3.0, AR-DPP 2.0, LDV 0.5, MI 0.1
YF damping δ	10^{-3}
Damping floor	10^{-6}
Feasibility tol. (λ_c EMA)	10^{-7}
Displacement tol. (λ_d EMA)	10^{-1}
EMA rate / max decades	10^{-2} / 1.0
Gradient clip norm	20.0

Table 7: Training hyperparameters. Unconditional models use the PAL projector; the conditional model uses the weighted-sum ramp (Section 5).

Appendix B Architecture comparison on converged seeds

Table 8 repeats the unconditional architecture comparison of Table 2 with the two diverged AR-Transformer seeds (Section 6) removed, so the AR rows are averaged over its eight converged seeds while the MLP and Parallel rows remain over all ten. This isolates the per-run behaviour of the AR architecture from its training-stability problem: on the converged seeds the AR-Transformer matches or slightly exceeds the Parallel decoder on most geometric metrics, confirming that the gap in Table 2 is an optimization-stability artifact rather than an architectural deficit.

This invites a natural question: if the AR instability were tuned away, or we simply kept a good seed, would the AR-Transformer become the preferred choice? On geometry the two attention models are close, which is expected, since they share the same DiT backbone, losses, and conditioning and differ only in decoding order. The recommendation still does not flip, because of latent usage: even on the converged seeds the AR model uses z less reliably and leans on its previously placed boxes (Section 6.6), most visibly at higher box counts. Closing this latent-usage gap is what would have to change before AR is the better fit for GLUE, and part of

that is understanding how the model keeps a reasonable MI while still using z weakly, since the MI loss is meant to prevent exactly that.

model	seq_len	Cleanup	Overlap ↓	VolAcc ↑	Compactness ↑	LDV ↓	MI ↓
MLP	3	off	0.009 ± 0.005	0.38 ± 0.01	0.41 ± 0.08	22.6 ± 0.5	3.8 ± 0.7
MLP	3	on	0.004 ± 0.001	**1.00 ± 0.00**	0.50 ± 0.04	22.4 ± 0.5	3.8 ± 0.7
MLP	10	off	0.094 ± 0.040	0.35 ± 0.01	0.41 ± 0.08	0.906 ± 0.517	4.1 ± 1.4
MLP	10	on	0.024 ± 0.003	**1.00 ± 0.00**	0.49 ± 0.02	0.314 ± 0.163	4.1 ± 1.4
AR-Transformer	3	off	0.006 ± 0.002	0.99 ± 0.01	0.62 ± 0.04	21.2 ± 0.6	**1.4 ± 0.1**
AR-Transformer	3	on	**0.003 ± 0.001**	**1.00 ± 0.00**	0.61 ± 0.04	21.2 ± 0.6	**1.4 ± 0.1**
AR-Transformer	10	off	0.082 ± 0.033	0.99 ± 0.01	**0.71 ± 0.06**	0.105 ± 0.060	1.6 ± 0.5
AR-Transformer	10	on	0.037 ± 0.013	**1.00 ± 0.00**	0.67 ± 0.07	**0.104 ± 0.073**	1.6 ± 0.5
Parallel	3	off	0.008 ± 0.003	0.98 ± 0.01	0.60 ± 0.05	21.1 ± 0.4	**1.4 ± 0.1**
Parallel	3	on	**0.003 ± 0.001**	**1.00 ± 0.00**	0.58 ± 0.04	21.1 ± 0.4	**1.4 ± 0.1**
Parallel	10	off	0.102 ± 0.028	0.98 ± 0.01	0.64 ± 0.04	0.143 ± 0.134	**1.4 ± 0.1**
Parallel	10	on	0.030 ± 0.011	**1.00 ± 0.00**	0.60 ± 0.03	0.129 ± 0.122	**1.4 ± 0.1**

Table 8: Architecture comparison with the two diverged AR-Transformer seeds excluded (AR averaged over its 8 converged seeds; MLP and Parallel over all 10). Otherwise identical to Table 2. Best value per column in bold (higher better for ↑, lower for ↓).